

模板

函数模板语法

```
#include<iostream>
using namespace std;

//template<typename T> + 函数声明或者定义
//template: 声明创建模板
//typename/class: 表明后面的符号是一种数据类型
//T: 通用的数据类型, 名称可以替换

//声明一个模板, 告诉编译器后面代码紧跟着的 T 不要报错, T是一个通用数据类型
template<typename T>

//函数模板
void myswap(T &a,T &b)
{
    T temp = a;
    a = b;
    b = temp;
}

template <typename T>
void func()
{
    cout<<"use func()"<<endl;
}

void test1()
{
    //模板必须确定 T 的数据类型才可以使用
    func<int>();
}

void test2()
{
    int a=10;
    int b=20;
    char c='c';

    //利用函数模板交换
    //1. 自动类型推导
    myswap(a,b);

    //自动类型推导必须推导出一致的数据类型 T 才可以使用
    //myswap(a,c); 错误

    //2. 显示指定类型
    myswap<int>(a,b); //<int>为模板的参数类型

    cout<<a<<" "<<b<<endl;
```

```

}

int main()
{
    test1();
    test2();

    return 0;
}

```

案例

```

#include<iostream>
using namespace std;

template<typename T>
void myswap(T &a,T &b)
{
    T temp=a;
    a=b;
    b=temp;
}

//选择排序
template<typename T>
void mysort(T arr[],int len)//T arr[]表示传一个数组
{
    for(int i=0;i<len;i++)
    {
        int max=i;
        for(int j=i+1;j<len;j++)
        {
            if(arr[j]>arr[max])
            {
                max=j;
            }
        }
        if(max!=i)
        {
            myswap(arr[max],arr[i]);
        }
    }
}

template<typename T>
void printarray(T arr[],int len)
{
    for(int i=0;i<len;i++)
    {
        cout<<arr[i];
    }
    cout<<endl;
}

```

```

void test1()
{
    char chararr[]="i love you";//定义一个字符类型的数组
    int num=sizeof(chararr)/sizeof(char);//chararr是该字符类型数组名
    mysort(chararr,num);
    printarray(chararr,num);
}

void test2()
{
    int intarr[]={2,5,3,7,8,5,9};
    int num=sizeof(intarr)/sizeof(int);
    mysort(intarr,num);
    printarray(intarr,num);
}

int main()
{
    test1();
    test2();

    return 0;
}

```

普通函数与函数模板区别

```

#include<iostream>
using namespace std;

//普通函数可以进行隐式类型转换
int myadd1(int a,int b)
{
    return a+b;
}

template<typename T>
T myadd2(T a,T b)
{
    return a+b;
}

void test()
{
    int a=10;
    int b=20;
    char c='c';//c的ASCII值是99
    cout<<myadd1(a,c)<<endl;//普通函数自动将字符型转换为整型
    //cout<<myadd2(a,c)<<endl;//函数模板 自动类型推导 不可以进行隐式转换
    cout<<myadd2<int>(a,c)<<endl;//函数模板 显示指定类型推导 可以进行隐式转换
}

```

```
int main()
{
    test();//109

    return 0;
}
```

普通函数与函数模板的调用规则

```
#include<iostream>
using namespace std;
//调用规则：
//1.函数模板与普通函数都可以实现时，优先调用普通函数
//2.可以通过空模板参数列表 强制调用函数模板
//3.函数模板可以发生重载
//4.如果函数模板可以产生更好的匹配，优先调用函数模板
void print(int a,int b)
{
    cout<<"调用普通函数"<<endl;
}

template<typename T>
void print(T a,T b)
{
    cout<<"调用的模板"<<endl;
}

template<typename T>
void print(T a,T b,T c)
{
    cout<<"调用重载的模板"<<endl;
}

void test()
{
    int a=10;
    int b=20;
    int c=30;
    //print(a,b);//优先调用普通函数模板
    print<>(a,b);//空模板参数列表，强制调用函数模板
    print(a,b,c); //普通函数与函数模板可以发生重载

    char m='m';
    char n='n';
    print(m,n); //函数模板可以产生更好的匹配，优先调用函数模板
    //底层逻辑：普通函数还需要类型转换 比较麻烦；函数模板直接推导出 T 就可以 比较简单
}

int main()
{
    test();
    return 0;
}
```

函数模板的局限性

```
#include<iostream>
#include<string>
using namespace std;
//对于特定的数据类型，需要用具体化方式做特殊实现
//解决方法： 1. 运算符重载 (==) 2.
template<typename T>
bool mycompare(T &a,T &b)
{
    if(a==b)
    {
        return true;
    }
    else
    {
        return false;
    }
}

class person{
public:
    person(string name,int age)
    {
        this->m_name=name;
        this->m_age=age;
    }

    string m_name;
    int m_age;
};

//利用具体化person的版本实现代码，具体化优先于模板调用
template<> bool mycompare(person &p1,person &p2)//前面加template<>表示具体化代码
{
    if(p1.m_age==p2.m_age&& p1.m_name==p2.m_name)
    {
        return true;
    }
    else
    {
        return false;
    }
}

void test()
{
    person p1("tom",10);
    person p2("tom",10);

    bool ret=mycompare(p1,p2);
    if(ret)
```

```

    {
        cout<<"两对象相等"<<endl;
    }
    else
    {
        cout<<"两对象不相等"<<endl;
    }
}

int main()
{
    test();

    return 0;
}

```

类模板基本语法

```

#include<iostream>
#include<string>
using namespace std;
//2.类模板在模板参数列表中可以有默认参数

template<class nametype,class agetype=int>
class person{
public:
    person(nametype name,agetype age)
    {
        this->m_name=name;
        this->m_age=age;
    }

    void showperson()
    {
        cout<<m_name<<" "<<m_age<<endl;
    }

    nametype m_name;
    agetype m_age;
};

void test()
{
    //1.类模板没有自动类型推到使用方式，只能用显式指定类型
    //person<string,int>p1("tom",22);//指定nametype为string类型，agetype为int类型
    person<string>p1("tom",22);
    p1.showperson();
}

int main()
{
    test();
}

```

```
    return 0;
}
```

类模板函数成员的创建时机

```
#include<iostream>
using namespace std;

class person1{
public:
    void showperson1()
    {
        cout<<"person1 show"<<endl;
    }
};

class person2{
public:
    void showperson2()
    {
        cout<<"person2 show"<<endl;
    }
};

template<class T>
class myclass{
public:
    //类模板中的成员函数不是一开始就创建的，而是在模板调用时再生成
    //如果一开始创建，编译器不能确认obj的类型
    void func1()
    {
        obj.showperson1();
    }
    void func2()
    {
        obj.showperson2();
    }

    T obj;
};

void test()
{
    myclass<person1>m;
    m.func1();
    //m.func2();
}

int main()
{
    test();

    return 0;
}
```

```
}
```

类模板对象做函数参数

```
#include<iostream>
#include<string>
using namespace std;
//类模板对象做函数参数
template<class T1,class T2>
class person{
public:
    person(T1 name,T2 age)
    {
        this->m_name=name;
        this->m_age=age;
    }

    void showperson()
    {
        cout<<this->m_name<<" "<<this->m_age<<endl;
    }
    T1 m_name;
    T2 m_age;
};

//1.指定传入类型
void printperson1(person<string,int> &p)
{
    p.showperson();
}

void test1()
{
    person<string,int>p("tom",22);
    printperson1(p);
}

//2.参数模板化
template<class T1,class T2>
void printperson2(person<T1,T2> &p)
{
    p.showperson();
}

void test2()
{
    person<string,int>p2("jerry",18);
    printperson2(p2);
}

//3.整个类模板化
```



```

template<class T>
void printperson3(T &p)
{
    p.showperson();
}

void test3()
{
    person<string,int>p3("kevin",33);
    printperson3(p3);
}

int main()
{
    test1();
    test2();
    test3();

    return 0;
}

```

类模板与继承

```

#include<iostream>
using namespace std;

template<class T>
class base{
public:
    T m;
};

class son1:public base<int>{ //必须要知道父类中 T 数据类型，才能继承给子类
public:

};
//如果想要灵活指定父类中 T 数据类型，子类也需要变类模板
template<class T1,class T2>
class son2:public base<T2>{//推导出父类中 T 为char数据类型
public:
    T1 obj;
};

void test1()
{
    son1 s1;
}

void test2()
{
    son2<int,char>s2;
}

```

```

int main()
{
    test1();
    test2();

    return 0;
}

```

类模板成员函数类外实现

```

#include<iostream>
#include<string>
using namespace std;

template<class T1,class T2>
class person{
public:
    person(T1 name,T2 age);
    void showperson();

    T1 m_name;
    T2 m_age;
};

template<class T1,class T2>
person<T1,T2>::person(T1 name,T2 age)//加上<T1,T2>表示是类模板的类外实现
{
    this->m_name=name;
    this->m_age=age;
}

template<class T1,class T2>
void person<T1,T2>::showperson()
{
    cout<<this->m_name<<" "<<this->m_age<<endl;
}

void test()
{
    person<string,int>p("zq",23);
    p.showperson();
}

int main()
{
    test();
    return 0;
}

```

类模板与友元

```
#include<iostream>
#include<string>
using namespace std;

template<class T1,class T2>
class person{

    template<class F1,class F2>
    friend void printperson(person<F1,F2> p);

public:
    person(T1 name,T2 age)
    {
        this->m_name=name;
        this->m_age=age;
    }

private:
    T1 m_name;
    T2 m_age;
};

template<class F1,class F2>
void printperson(person<F1,F2> p)//参数模板化
{
    cout<<p.m_name<<" "<<p.m_age<<endl;
}

void test()
{
    person<string,int>p("tom",22);
    printperson(p);
}

int main()
{
    test();

    return 0;
}
```